

Hands-on 4: Difference between JPA, Hibernate, and Spring Data JPA

1. Java Persistence API (JPA)

- JPA is a **Java specification** for Object Relational Mapping (ORM).
- It defines a standard way to map Java objects to database tables.
- JPA itself **does not provide an implementation**.
- It only defines interfaces and annotations like:
 - @Entity
 - @Table
 - @Id
 - @Column

Examples of JPA implementations:

- Hibernate
- EclipseLink
- OpenJPA

2. Hibernate

- Hibernate is an **ORM framework**.
- It is the **most popular implementation of JPA**.
- It converts Java objects into database records automatically.
- Hibernate provides many additional features beyond JPA.

Advantages

- Automatic SQL generation
- Caching support
- Lazy loading
- HQL (Hibernate Query Language)

3. Spring Data JPA

- Spring Data JPA is built on top of JPA.
- It **does not implement JPA**.
- It uses Hibernate (or another JPA provider) internally.
- It greatly reduces boilerplate code.
- It automatically provides CRUD operations.

Instead of :

```
Session session = factory.openSession();
Transaction tx = session.beginTransaction();
session.save(employee);
tx.commit();
session.close();
```

We can:

```
employeeRepository.save(employee);
```

Comparison Table :

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	Framework	Spring Module
ORM Support	Defines ORM rules	Implements ORM	Uses JPA implementation
Database Operations	No	Yes	Yes
Transaction Management	No	Manual	Automatic (@Transactional)
Boilerplate Code	High	Medium	Very Low
CRUD Methods	No	Manual	Built-in
Query Support	JPQL	HQL & JPQL	Method Name Queries, JPQL

Hibernate Example

```
public Integer addEmployee(Employee employee) {  
  
    Session session = factory.openSession();  
    Transaction tx = null;  
    Integer employeeID = null;  
  
    try {  
  
        tx = session.beginTransaction();  
  
        employeeID = (Integer) session.save(employee);  
  
        tx.commit();  
  
    } catch (HibernateException e) {  
  
        if (tx != null)  
            tx.rollback();  
  
        e.printStackTrace();  
  
    } finally {  
  
        session.close();  
  
    }  
  
    return employeeID;  
}
```

Spring Data JPA Example

Repository

```
public interface EmployeeRepository extends JpaRepository<Employee,  
Integer> {  
  
}
```

Service

```
@Service  
public class EmployeeService {  
  
    @Autowired  
    private EmployeeRepository employeeRepository;  
  
    @Transactional  
    public void addEmployee(Employee employee) {  
  
        employeeRepository.save(employee);  
  
    }  
}
```